

RIPHAH INTERNATIONAL COLLEGE HARBANSPURA CAMPUS

A Project of Riphah International University

ADP (ITM) — Artificial Intelligence

Industry-Ready Student Project Catalog

Part 1 of 2 | Projects 1–10

Semester: 3rd | Course: Introduction to Artificial Intelligence

Program: ADP (ITM) | Credit Hours: 3

Introduction

This catalog presents 10 industry-ready project ideas (Part 1 of 2) designed for ADP (ITM) students enrolled in the Introduction to Artificial Intelligence course at Riphah International Colleges. Each project has been carefully curated to align with the official course outline and its defined Course Learning Outcomes (CLOs).

All projects are scoped to be realistically completable within a 6–8 week development window, making them suitable for semester-end project assignments. Each entry includes a project title, category, abstract, detailed functional requirements, and recommended tools and technologies.

Guidelines for Students

- Each student or group must select one unique project title.
- Students must develop a working prototype that satisfies all listed functional requirements.
- All tools and technologies listed are recommendations; alternatives may be used with instructor approval.
- A project demonstration and report submission is mandatory at the end of the semester.
- Projects must include a short presentation of 5–10 minutes covering objectives, methodology, and results.

CLO Alignment Summary

Each project specifies which CLOs it covers. Students should ensure their project implementation satisfies the mapped CLO requirements as defined in the course outline.

Project 1: AI-Powered Spam Email Classifier

Category	Development Timeline	Difficulty Level
Machine Learning / NLP	6–8 Weeks	Intermediate

CLO Coverage: CLO2, CLO3, CLO4, CLO5

Abstract

This project involves building an intelligent email spam detection system using classical Machine Learning algorithms and Natural Language Processing (NLP) techniques. Students will process raw email datasets, extract meaningful text features using Bag-of-Words and TF-IDF vectorization, and train multiple classifiers including Naive Bayes and K-Nearest Neighbors to distinguish between spam and legitimate emails. The final product is a working web-based or desktop interface where a user can input email text and receive an instant prediction. This project directly aligns with the course units on NLP, Naive Bayes classification, Bag of Words, Tokenization, and Supervised Learning.

Detailed Functional Requirements

1. The system shall allow users to input raw email text through a simple text-box interface.
2. The system shall preprocess input text by performing tokenization, stop-word removal, and stemming.
3. The system shall apply TF-IDF or Bag-of-Words vectorization to convert text into numerical feature vectors.
4. The system shall train and evaluate at least two classifiers: Naive Bayes and KNN, comparing their accuracy.
5. The system shall display classification result (Spam / Not Spam) with a confidence percentage.
6. The system shall generate a classification report including precision, recall, and F1-score.
7. The system shall visualize word frequency distribution using bar charts or word clouds.
8. The system shall support loading a custom email dataset (CSV format) for retraining.
9. The system shall store training history and model accuracy in a local log file.
10. The system shall provide a confusion matrix visualization for model performance review.

Tools and Technologies

Python 3.x, Scikit-learn, NLTK, Pandas, NumPy, Matplotlib, Streamlit or Tkinter, VS Code / PyCharm

Project 2: Intelligent Student Performance Prediction System

Category	Development Timeline	Difficulty Level
Machine Learning / Supervised Learning	6–8 Weeks	Intermediate

CLO Coverage: CLO2, CLO4, CLO5

Abstract

This project focuses on predicting a student's academic performance based on demographic, behavioral, and academic input features using supervised machine learning techniques. Students will work with a structured dataset containing attributes such as attendance, assignment scores, study hours, and previous grades. Multiple regression and classification algorithms will be applied, including KNN and Naive Bayes classifiers, to predict final grades or risk categories (Pass/Fail/At-Risk). The project aligns with the course units on Supervised Learning, KNN, Dataset Preparation, and Model Evaluation.

Detailed Functional Requirements

1. The system shall accept student academic and behavioral data through a form-based interface.
2. The system shall preprocess data by handling missing values, normalizing numerical features, and encoding categorical variables.
3. The system shall train KNN and at least one additional classification algorithm on a student dataset.
4. The system shall split data into training (70%), validation (15%), and test (15%) sets.
5. The system shall display the predicted performance category (Excellent / Pass / At Risk / Fail).
6. The system shall generate model accuracy, precision, recall, and F1-score on test data.
7. The system shall visualize feature importance and correlation heatmaps.
8. The system shall allow educators to input new student data and receive a real-time prediction.
9. The system shall display historical prediction logs for multiple student records.
10. The system shall compare performance of different classifiers using a bar chart visualization.

Tools and Technologies

Python 3.x, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, Streamlit or Flask, PyCharm / VS Code

Project 3: AI-Based Disease Diagnosis Assistant (Symptom Checker)

Category	Development Timeline	Difficulty Level
Machine Learning / Expert Systems	7–8 Weeks	Intermediate

CLO Coverage: CLO3, CLO4, CLO5

Abstract

This project develops a rule-based and ML-powered medical symptom checker that assists in preliminary disease diagnosis. Students will implement a knowledge base containing symptom-disease relationships and use both forward chaining inference (mimicking an expert system) and a trained Naive Bayes classifier for probabilistic diagnosis. The system will accept user-entered symptoms and return the most probable disease along with recommended actions. This project maps directly to course units on Expert Systems, Knowledge Base, Inference Engine, Forward Chaining, and Naive Bayes Classification.

Detailed Functional Requirements

1. The system shall present users with a checklist of common symptoms to select from.
2. The system shall use a rule-based inference engine (forward chaining) to match symptoms to diseases.
3. The system shall apply a trained Naive Bayes classifier as a secondary diagnosis mechanism.
4. The system shall output the top 3 most probable diseases based on entered symptoms.
5. The system shall display confidence scores for each diagnosed condition.
6. The system shall provide a basic description and recommended action for each diagnosed disease.
7. The system shall allow administrators to add new rules and diseases to the knowledge base.
8. The system shall generate a session report summarizing entered symptoms and diagnosis results.
9. The system shall handle ambiguous symptoms by requesting additional clarification questions.
10. The system shall include a disclaimer clarifying it is not a replacement for professional medical advice.

Tools and Technologies

Python 3.x, Scikit-learn (Naive Bayes), Pandas, JSON (Knowledge Base), Streamlit or Tkinter, PyCharm

Project 4: Pathfinding Visualizer using AI Search Algorithms

Category	Development Timeline	Difficulty Level
AI Search Algorithms / Problem Solving	6–7 Weeks	Intermediate

CLO Coverage: CLO2, CLO3

Abstract

This project creates an interactive graphical application that visualizes how different AI search algorithms navigate through a grid-based maze to find the shortest path from a source to a destination. Students will implement and compare Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search, and A* Search algorithms. A visual interface will display the exploration frontier, visited nodes, and the final path. This project aligns with course units on Search Strategies, BFS, DFS, Uniform Cost Search, Iterative Deepening, and A* Search.

Detailed Functional Requirements

1. The system shall display an interactive $N \times N$ grid where users can place walls, start, and end points.
2. The system shall implement BFS, DFS, Uniform Cost Search, Iterative Deepening, and A* algorithms.
3. The system shall animate the search process step-by-step showing nodes being explored in real time.
4. The system shall highlight the final optimal path from start to destination using a distinct color.
5. The system shall display metrics including total nodes explored, path length, and time taken.
6. The system shall allow users to switch between algorithms and compare results on the same grid.
7. The system shall allow users to set custom start and end points via mouse click.
8. The system shall support randomized maze generation to create new test scenarios.
9. The system shall allow controlling animation speed (slow/medium/fast).

10. The system shall display a side-by-side comparison table of algorithm performance after execution.

Tools and Technologies

Python 3.x, Pygame or Tkinter, Collections (deque), PriorityQueue, NumPy, PyCharm / VS Code

Project 5: Handwritten Digit Recognition using Artificial Neural Networks

Category	Development Timeline	Difficulty Level
Machine Learning / Artificial Neural Networks	7–8 Weeks	Intermediate

CLO Coverage: CLO3, CLO4, CLO5

Abstract

This project implements an Artificial Neural Network (ANN) from scratch (using NumPy) and also using Scikit-learn's MLPClassifier to recognize handwritten digits from the MNIST dataset. Students will explore forward propagation, back propagation, activation functions, and tuning of neural network hyperparameters such as learning rate and number of hidden layers. A visual interface will allow users to draw a digit and receive a real-time prediction. This project aligns directly with the course topics on ANN Model Representation, Activations, Bias, Forward and Back Propagation, and Complexity Estimation.

Detailed Functional Requirements

1. The system shall load and preprocess the MNIST dataset (28×28 pixel images, normalized to 0–1).
2. The system shall implement forward propagation with configurable hidden layers and activation functions (ReLU, Sigmoid).
3. The system shall implement backpropagation for weight update using gradient descent.
4. The system shall train the ANN and display training accuracy and loss per epoch via a graph.
5. The system shall achieve at least 90% test accuracy on the MNIST test set.
6. The system shall provide a canvas interface where users can draw a digit using the mouse.
7. The system shall perform real-time prediction on the drawn digit and display the result.
8. The system shall display top-3 predicted digits with their confidence percentages.

9. The system shall allow adjusting hyperparameters (learning rate, epochs, hidden units) through the UI.
10. The system shall generate a model evaluation report including confusion matrix and accuracy per digit class.

Tools and Technologies

Python 3.x, NumPy, Scikit-learn (MLPClassifier), Matplotlib, Tkinter or Streamlit, MNIST Dataset, PyCharm

Project 6: AI Tic-Tac-Toe Game using Minimax Algorithm

Category	Development Timeline	Difficulty Level
AI Game Programming / Adversarial Search	5–6 Weeks	Intermediate

CLO Coverage: CLO2, CLO3

Abstract

This project implements an unbeatable Tic-Tac-Toe game where the computer opponent is powered by the Minimax adversarial search algorithm. Students will develop the game logic, the search tree, and a graphical user interface for human-vs-computer gameplay. An optional Alpha-Beta pruning enhancement will be explored to optimize the algorithm's search efficiency. This project directly maps to the course units on Adversarial Search, Minimax Algorithm, Game Programming, and Problem Solving Agents.

Detailed Functional Requirements

1. The system shall implement a fully functional Tic-Tac-Toe game with a graphical 3×3 board interface.
2. The system shall implement the Minimax algorithm that allows the AI to make optimal moves.
3. The system shall allow the user to choose to play as X or O before the game starts.
4. The system shall allow the user to choose difficulty level: Easy (random AI), Medium, or Hard (Minimax).
5. The system shall detect and announce win, loss, or draw conditions after each game.
6. The system shall visualize the Minimax decision tree (partial) in a side panel for educational purposes.

7. The system shall implement Alpha-Beta pruning as an option and display nodes pruned vs explored.
8. The system shall track and display session scores (wins, losses, draws) across multiple games.
9. The system shall support a two-player human mode as an alternative game option.
10. The system shall include a reset button to start a new game at any point during gameplay.

Tools and Technologies

Python 3.x, Pygame or Tkinter, PyCharm / VS Code

Project 7: Sentiment Analysis of Product Reviews using NLP

Category	Development Timeline	Difficulty Level
Natural Language Processing / Machine Learning	6–8 Weeks	Intermediate

CLO Coverage: CLO2, CLO4, CLO5

Abstract

This project builds a sentiment analysis engine that classifies customer product reviews as Positive, Negative, or Neutral using NLP preprocessing techniques and supervised machine learning classifiers. Students will work with a real-world review dataset (e.g., Amazon or movie reviews), apply tokenization, stemming, stop-word removal, and Bag-of-Words features, and train Naive Bayes and KNN classifiers. A dashboard will display sentiment distribution charts and keyword analysis. This project aligns with course units on NLP, Sentiment Analysis, Bag of Words, Tokenization, Stop Words, Naive Bayes, and Data Visualization.

Detailed Functional Requirements

1. The system shall accept a CSV file containing product review text and a sentiment label column.
2. The system shall preprocess review text including tokenization, stop-word removal, and stemming.
3. The system shall apply Bag-of-Words and TF-IDF vectorization to extract features.
4. The system shall train and compare Naive Bayes and KNN classifiers on the prepared dataset.
5. The system shall classify each review as Positive, Negative, or Neutral.

6. The system shall display overall model accuracy, precision, recall, and F1-score.
7. The system shall generate a pie chart and bar chart showing sentiment distribution across the dataset.
8. The system shall allow a user to type a custom review and receive instant sentiment prediction.
9. The system shall display a word cloud for positive and negative review keywords separately.
10. The system shall export the classified review results to a CSV file for download.

Tools and Technologies

Python 3.x, NLTK, Scikit-learn, Pandas, Matplotlib, WordCloud library, Streamlit or Flask, PyCharm

Project 8: K-Means Customer Segmentation System

Category			Development Timeline	Difficulty Level
Machine Learning	/	Unsupervised Learning	6–7 Weeks	Intermediate

CLO Coverage: CLO4, CLO5

Abstract

This project applies the K-Means clustering algorithm to segment customers based on their purchasing behavior, demographics, or usage patterns. Students will work with a retail or e-commerce dataset, preprocess the data, determine the optimal number of clusters using the Elbow Method, and visualize the resulting customer segments in 2D space using PCA dimensionality reduction. The system will help businesses understand distinct customer groups for targeted marketing. This project aligns with course units on Clustering (K-Means), Dimensionality Reduction (PCA), Unsupervised Learning, and Data Visualization.

Detailed Functional Requirements

1. The system shall accept a customer dataset in CSV format with numerical feature columns.
2. The system shall preprocess data by handling missing values, normalizing features, and encoding categorical fields.
3. The system shall implement the K-Means clustering algorithm with configurable number of clusters (K).

4. The system shall apply the Elbow Method to recommend the optimal number of clusters automatically.
5. The system shall apply PCA to reduce dataset dimensions to 2D for visualization purposes.
6. The system shall display a 2D scatter plot of customer clusters with distinct color coding.
7. The system shall generate a cluster summary table showing the centroid values and cluster size.
8. The system shall assign meaningful segment labels (e.g., High Value, Low Engagement) based on centroid analysis.
9. The system shall allow users to upload a new dataset and re-run clustering dynamically.
10. The system shall export cluster assignments appended to the original dataset as a downloadable CSV.

Tools and Technologies

Python 3.x, Scikit-learn (KMeans, PCA), Pandas, NumPy, Matplotlib, Seaborn, Streamlit or Jupyter Notebook, PyCharm

Project 9: AI Knowledge Base Chatbot using Prolog

Category	Development Timeline	Difficulty Level
Knowledge Representation / Prolog / Expert Systems	6–7 Weeks	Intermediate

CLO Coverage: CLO2, CLO3, CLO5

Abstract

This project develops a rule-based chatbot powered by Prolog's logic programming paradigm. Students will define a knowledge base containing facts and rules about a specific domain (e.g., university information, country facts, or a career advisor system), and use first-order predicate logic with backward chaining to answer user queries. The Prolog engine will be integrated with a Python-based interface for user interaction. This project directly aligns with course topics on First Order Logic (FOL), Prolog, Inference Rules, Backward Chaining, and Knowledge Representation.

Detailed Functional Requirements

1. The system shall implement a structured Prolog knowledge base with at least 50 domain-specific facts.
2. The system shall define logical rules using Prolog clauses to derive new knowledge from existing facts.

3. The system shall support natural language-style user queries through a Python terminal or GUI interface.
4. The system shall use Prolog's backward chaining mechanism to answer queries from the knowledge base.
5. The system shall handle multi-step reasoning queries requiring more than one inference step.
6. The system shall display the inference chain (proof path) for each answered query.
7. The system shall return a 'not found' or 'unknown' response for queries outside the knowledge base.
8. The system shall support adding new facts to the knowledge base dynamically at runtime.
9. The system shall log all queries and responses in a session history file.
10. The system shall demonstrate at least 3 different types of logical rules (inheritance, classification, recommendation).

Tools and Technologies

SWI-Prolog, Python 3.x (PySwip library for Python-Prolog integration), Tkinter or terminal, VS Code

Project 10: Real-Time Face Emotion Detection System

Category	Development Timeline	Difficulty Level
Machine Learning / Computer Vision / NLP	7–8 Weeks	Intermediate

CLO Coverage: CLO1, CLO4, CLO5

Abstract

This project builds a real-time facial emotion detection application using a pre-trained deep learning model integrated with OpenCV for live camera feed processing. Students will capture video frames, detect faces using Haar Cascade classifiers, preprocess face ROIs, and pass them through a trained ANN or CNN-based emotion classifier to recognize emotions such as Happy, Sad, Angry, Surprised, and Neutral. The project aligns with course units on Image Processing, Artificial Neural Networks, Introduction to Deep Learning, and Generative AI overview.

Detailed Functional Requirements

1. The system shall capture live video input from the device's webcam in real time.

2. The system shall detect human faces in each video frame using OpenCV's Haar Cascade classifier.
3. The system shall preprocess detected face regions (grayscale conversion, resizing to 48×48 pixels, normalization).
4. The system shall classify each detected face into one of 7 emotion categories: Happy, Sad, Angry, Fear, Disgust, Surprise, Neutral.
5. The system shall display the detected emotion label and confidence score overlaid on the video frame.
6. The system shall support detection of multiple faces simultaneously in a single frame.
7. The system shall display an emotion frequency bar chart updating in real time during the session.
8. The system shall allow capturing a snapshot of a frame and saving the annotated image to disk.
9. The system shall display session statistics including dominant emotion and detection accuracy summary at session end.
10. The system shall support running in image mode (single uploaded photo) in addition to live camera mode.

Tools and Technologies

Python 3.x, OpenCV, TensorFlow / Keras (pre-trained model), NumPy, Matplotlib, Streamlit or Tkinter, FER2013 Dataset, PyCharm